The Software Factory

How an idea becomes a plan, a plan becomes merged code, and what the system learns on the way back.
Four parts.

01

Planning

artifacts · hand-over · loop

02

Implementation

done · tdd · review

03

Operate & Learn

check · record · feed back

04

Knowledge base

graph · build · proof

Planning produces two artifacts, and hands over tickets

A prototype at an agreed level of maturity, a plan everyone can work from, and tickets the factory can turn into failing tests.

`prototype` `pm` `po` `exec`

"Agreed level of maturity" prototype

The maturity is agreed up front, so an idea is tested with little or no developer time.

`click-dummy` simple click dummy with fake data · `running-prototype` partially connected, actually running

`plan` `pm` `po` `the factory`

The plan

Written in a strong format that we provide, so every plan looks the same and can be picked up by anyone, or by the factory.

`versioned` every change is a new version · `strong-format` a fixed template from us · `collaborative` written and revised together

THE TICKET THE FACTORY READS `THE FACTORY`

```
# Ticket
> the task in one sentence
## Done when these pass
- [ ] one to five acceptance criteria, each testable
      more than five: split the ticket
## Affects
- services, owners and consumers (from the knowledge base)
## Rules
- conventions and decisions that apply
## Similar
- past changes to learn from, with what they cost
```

> **Sized for the factory.** A ticket that cannot become tests, or that runs out of rounds, comes back as a planning finding. Content comes from the knowledge base, from Operate & Learn, and from the people in the room, asked now and not later.

From plan to tickets to the factory, and back

The plan is broken into tickets and the factory implements them. Failures come back to the plan first; what happened in production comes back through Operate & Learn.

plan

Plan part 01

The planning artifact. Everything below the dashed line is derived from it.

tickets

Tickets derived

The plan is broken down into tickets, each sized for one run.

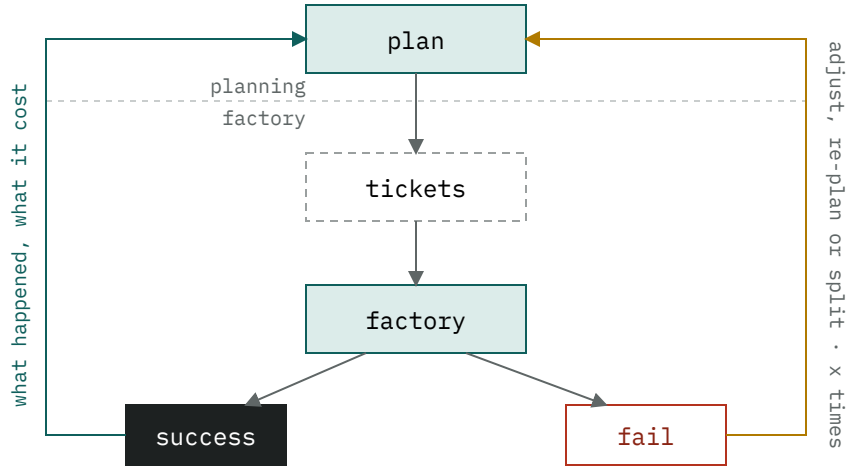
factory

Implementation the factory

Each ticket goes through the implementation loop. It ends in success or in a fail.

WHAT COMES BACK

- ✓ Success: Operate & Learn checks whether we asked for the right thing, and feeds the next plan.
- ↻ Fail: the plan is adjusted and the ticket re-planned or split, x times. The factory's own retries stay inside the factory.
- ✗ Then it is a blocker: a human takes over.



After x times

the loop stops adjusting

`Blocker` The ticket is a blocker.

`Human` A human takes over.

Define done as failing tests, then one round per agent

The ticket is turned into acceptance tests that fail today. When they pass, the ticket is finished and nothing more is owed. Each later visit is a fresh agent doing one round, and the PR's build grades it, never the agent.

`define-done`

Write the red acceptance tests agent

Reads the ticket and any spec it points to. Writes the fewest tests that mean "done": one is ideal, more than five means the ticket needs splitting. Proves each fails because the behaviour is missing. Writes no production code. Picks exactly one strategy: `direct` · `parallel-change` · `characterize` · `mechanical`.

`open-pr`

Push and open a draft PR agent

Pushes the tests and the `dod.md` brief, then opens a draft pull request, which gives CI something to judge and keeps code review quiet.

`tdd-round`

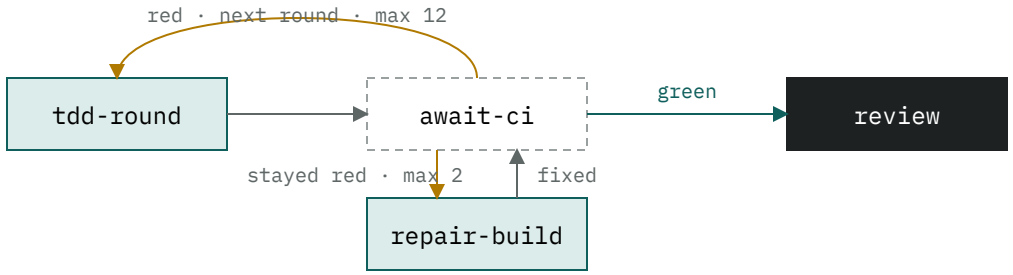
One red-green-refactor round agent

Writes one failing test for the smallest open facet, the least code that passes it, then refactors. Ticks the facet and pushes. Never edits an acceptance test to make it pass.

`repair-build`

Repair a stuck build agent

Used when the build stayed red and the round pushed nothing. Gets the failing checks from CI and fixes only those.



> **The agent doesn't run lint, typecheck or the full suite.** CI already runs all of it on every push, so the agent runs only the tests it's working on.

> **Twelve rounds is the limit on purpose.** A ticket that needs more was sized wrong, so running out is itself the finding.

LEAVES THIS STAGE

- → CI is green: on to review and approval.
- ✗ The ticket can't be expressed as tests, out of rounds, or a repair failed: blocked, with the reason on the ticket.
- ✓ The ticket is already resolved: ticket and draft PR are closed with the reason.

Out of draft, reviewed, merged by risk

The PR gets a real description and leaves draft, which starts code review. Once it is ready, two questions decide whether the factory merges it or a human does.

`ready-for-review`

Describe the finished work agent

Reads the diff against `dod.md` and writes the PR description and a new title, including why it strayed from the planned strategy, if it did. Removes `dod.md` and takes the PR out of draft.

`await-pr`

Wait for CI and review waits on CI

Resumes when CI is green on the latest commit and every review thread is resolved.

`fix-ci`

Fix a red build agent · max 3

Starts from CI's report: the failing checks, their annotations and the failed step. Fixes only those, pushes, and goes back to waiting.

ONCE THE PR IS READY (GREEN CI, NO OPEN THREADS), WHO MERGES IT?

1 Are the lights off? `lights-out mode, set per repository`	`Off` No humans in the loop. The factory merges it automatically. `On` Go to the risk assessment.
2 How risky is this PR? `a prompt reads the ready PR`	`Low` The factory merges it automatically. `High` A human reviews and merges it.

Never ready

The build is still red after 3 fixes, or review threads stay unresolved. The ticket is labeled `blocked` and gets a comment with the conclusions. The PR stays open for a human.

The check that runs after release, and what it feeds

Acceptance criteria say the software does what we asked for. This check says we asked for the right thing. Its conclusions, not the data, go into the knowledge base and into the next plan.

WHAT HAPPENS AFTER RELEASE

The check runs on its own	Worked · didn't work · can't tell · something got worse
People decide what to do	The check gives a result, not a decision
If something got worse, the system acts	Flag off, roll back. That is safety, not judgement

SIX THINGS EACH CHANGE DECLARES IN ITS PLAN

`signal` the one measure that shows whether the change worked

`who` who the change applies to

`guardrail` what must not get worse

`how-far` how much the signal should move

`how-long` how long we watch before calling it

`attribution` how we tell this change apart from the rest

SAVED TO THE KNOWLEDGE BASE, WITHOUT FLOODING IT

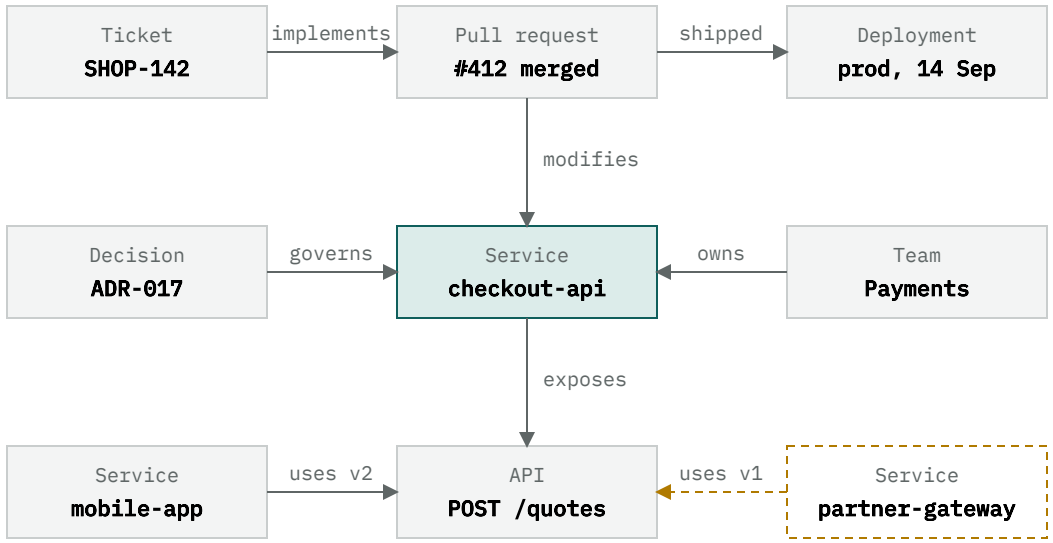
When something happens	A release, a merge, an incident, a result
Regular summaries	One per component, never the raw stream
Only if it says something new	"Nothing changed" is not worth saving
`written by people`	Why we chose this, what a result means, what went wrong. The factory asks at the time; nobody writes it up later

WHAT IT CHANGES FOR PLANNING

How past changes turned out	Targets are argued from what happened before
Changes that did not work	Come back as new work, with the evidence attached
Things that worked more than once	Become a starting template
What past work cost and took	Says what this one will likely cost, before we commit

A shared memory for the software factory

Agents can read the repository in front of them. The knowledge base holds everything else the company knows: other services, tickets, decisions, deployments, owners and rules, connected across every repo. Local code always wins.



solid = parsed from source data · dashed amber = inferred by AI, with evidence

LAYER	WHAT IT DOES
Sources	Git host, tickets, docs, CI/CD, ownership, incidents, release check results and factory run records
Ingestion	Read-only connectors, secrets and personal data removed, kept current by webhooks and a nightly sync
Processing	Cross-repo facts linked through shared IDs; AI summaries and decisions only with evidence; conclusions, never raw monitoring data
Storage	Knowledge graph and text search, plus versioned plans, specs, decisions and conventions in Git
Context service	One endpoint, six tools: task context, consumers, owners, conventions, similar changes, past outcomes. Permission filtering and an audit log
Consumers	Planning, the implementation agents, the risk assessment and the release check all query the same service; agents also get context at session start

> **Why agents can trust it.** Every fact records its source, whether it is approved, parsed or AI-inferred, and when it stopped being true. Across every layer: a model gateway to the customer's approved LLM, mirrored permissions, one isolated instance per customer.

How the factory uses it, and how we prove it works

Every factory step reads the same knowledge base, and every step writes back. Humans approve knowledge the same way they approve code: through pull requests.

AT EVERY FACTORY STEP

Planning	Past outcomes, caveats, cost and time of similar work
Define done	The spec, consumers and similar tickets shape the red acceptance tests
TDD rounds	Context pack, wiki pages and the conventions that apply
Risk check	Consumers and owners of the changed code feed the risk prompt
Release check	Uses the signals declared in the plan; writes conclusions back
Learn	Runs and human context flow back; owners approve inferred facts

ROLLOUT

1 Readiness check | 2 Code and ownership | 3 Tickets and docs | 4 AI enrichment | 5 Full factory integration

WHAT THE AI AGENT RECEIVES FOR SHOP-142

```
Task      Add discount codes to quotes
          Invalid code returns 422, one code per quote
Affects   checkout-api (owner: Payments)
          POST /quotes, used by mobile-app v2
          partner-gateway v1 (inferred, 0.8)
Rules     Money in integer cents; ADR-017 rounding
Similar   #398 added a field in v2 only
Past      #398 took 4 rounds; release check: worked
```

HOW WE PROVE IT

Readiness check	Of the customer's data before building
Question set	Real questions scored every night
With and without	The knowledge base on the same tickets: correctness, rework, time and cost